

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA5112 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5

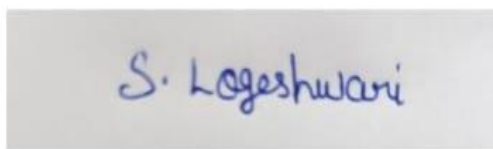
TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

I _Logeshwari S (192565040) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in blue ink that reads "S. Logeshwari".

QUESTION 1 – WEAK PASSWORD HASHING IN WEB APPLICATION

A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables. Analyze how the lack of salting and use of SHA-1 contributed to this vulnerability. Recommend a secure password hashing strategy and justify your choice.

Answer

The vulnerability is caused by two major weaknesses in the password-storage design: the use of unsalted hashes and the use of SHA-1 for password hashing. Although hashing is a one-way operation, a general-purpose cryptographic hash such as SHA-1 is not designed to make password guessing expensive. When it is also used without a unique salt, identical passwords produce identical hash values, making large-scale cracking much easier.

Impact of the Lack of Salting

- A salt is a unique, randomly generated value stored with each password hash. It ensures that the same password does not produce the same stored hash for different users.
- Because the application uses unsalted hashes, two users who choose the same password will have exactly the same SHA-1 hash.
- Attackers can use precomputed lookup tables, commonly called rainbow tables, to compare stolen hashes against previously computed password hashes.
- The absence of unique salts also makes bulk attacks efficient because one computed candidate password can be compared with many accounts at once.

- A salt does not need to be secret; its purpose is to make each password-hashing operation unique and defeat precomputation.

Problems with Using SHA-1 for Passwords

- SHA-1 is a fast general-purpose hash function. Password hashing requires the opposite property: each password guess should be deliberately expensive.
- Modern attackers can test very large numbers of password guesses using powerful hardware, especially when the password hash function is fast.
- SHA-1 is also considered obsolete for collision resistance. Although collision resistance is not the same issue as password cracking, its outdated security status is another reason not to select SHA-1 for a new password-storage system.
- Therefore, using SHA-1 directly for passwords provides insufficient resistance against offline password-guessing attacks after a database breach.

Recommended Secure Password-Hashing Strategy

The application should replace unsalted SHA-1 with a modern password-hashing algorithm such as Argon2id. If Argon2id is unavailable, a properly configured bcrypt or scrypt implementation can be considered. The system should automatically generate a unique random salt for every password and store the salt together with the resulting password hash and the algorithm parameters.

- Use Argon2id as the preferred password-hashing algorithm.
- Generate a cryptographically secure, unique random salt for every password.
- Use a deliberately expensive configuration involving appropriate memory, time, and parallelism settings.
- Store the resulting hash, salt, and necessary algorithm parameters in the database; the salt itself does not have to be secret.
- Never store passwords in plaintext and never use a fast general-purpose hash such as SHA-1 directly for password storage.

- During login, retrieve the stored parameters, hash the supplied password using the same configuration, and compare the result using a secure verification routine.
- Use rate limiting and account-protection controls to reduce online password-guessing attempts.

Justification

Argon2id is appropriate because it is specifically designed for password hashing and can be configured to require significant computational and memory resources. A unique salt prevents precomputed tables from being reused across accounts, while the deliberately expensive hashing process makes large-scale offline guessing substantially more difficult. Together, salting, memory-hard password hashing, secure parameters, and additional authentication controls provide a much stronger password-storage strategy than unsalted SHA-1.

Conclusion:

The breach was made significantly more dangerous because identical passwords generated identical unsalted SHA-1 hashes and the hashing process was too fast for password protection. Replacing SHA-1 with a modern password-hashing scheme such as Argon2id and using a unique salt for every password is the recommended solution.

QUESTION 2 – DIGITAL SIGNATURE COMPROMISE IN FINANCIAL SYSTEM

A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions. Evaluate how the compromise affects authentication and non-repudiation. Propose mitigation strategies to secure private keys and prevent such incidents.

Answer

A digital signature provides evidence that a transaction was signed using the corresponding private key and allows the recipient to verify the signature using the public key. Therefore, compromise of the private key is a serious security incident because the attacker can generate valid-looking signatures for unauthorized transactions.

Effect on Authentication

- Normally, successful signature verification indicates that the holder of the corresponding private key approved the transaction.
- If an attacker obtains the legitimate private key, the attacker can create signatures that verify correctly under the user's public key.
- The system may therefore incorrectly authenticate the attacker as the legitimate key owner.
- This breaks the intended binding between the user's identity and actual control of the private key.
- The incident should be treated as a credential/key compromise rather than as a simple invalid-signature event.

Effect on Non-Repudiation

Non-repudiation means that a valid digital signature can provide evidence linking a transaction to the corresponding signing key. However, if the private key was compromised, the original user may reasonably dispute having created the unauthorized transaction. Consequently, the evidentiary value of signatures created after the compromise becomes questionable unless the institution can establish when the key was compromised, when it was revoked, and whether additional transaction controls were satisfied.

Mitigation Strategies

- Protect private keys using hardware security modules (HSMs) or other hardware-backed secure key-storage mechanisms where appropriate.
- Use strong access controls and multi-factor authentication for administrative and signing operations.
- Never expose private keys unnecessarily or store them in plaintext files.
- Apply least-privilege principles so that only authorized signing components can use the key.
- Use secure key-generation procedures and maintain a documented key lifecycle covering generation, storage, rotation, backup, revocation, and destruction.
- Monitor signing activity and establish alerts for unusual transaction amounts, locations, times, frequencies, or destinations.
- Use transaction authorization controls such as multi-person approval for high-value transactions.
- Maintain tamper-evident audit logs so suspicious signing activity can be investigated.

- Immediately revoke or invalidate the compromised certificate/key and issue a replacement key pair.
- Use certificate revocation mechanisms such as CRLs or OCSP where applicable, and ensure relying systems check certificate status.

Incident Response

- Identify and confirm the private-key compromise.
- Suspend or restrict the affected signing credential.
- Revoke the associated certificate and prevent further use of the compromised key.
- Investigate all transactions signed during the suspected compromise period.
- Generate a new key pair using a secure process and issue a new certificate.
- Review logs, access controls, endpoint security, and key-management procedures to determine the cause.
- Strengthen transaction approval and monitoring controls before restoring normal signing operations.

Conclusion:

A compromised private key can undermine both authentication and the practical non-repudiation value of signatures made with that key. Strong private-key protection, rapid revocation, hardware-backed key storage, monitoring, multi-factor controls, and transaction-level authorization are essential.

QUESTION 3 – KERBEROS AUTHENTICATION FAILURE IN DISTRIBUTED SYSTEM

A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers. Analyze how time synchronization impacts Kerberos functionality. Suggest solutions to ensure reliable authentication and maintain system security.

Answer

Kerberos is a ticket-based authentication protocol that uses time-sensitive credentials. A client obtains authentication information from the Key Distribution Center (KDC) and then uses tickets to access network services. Time synchronization is therefore an important operational requirement.

Time Synchronization Matters

- Kerberos tickets contain timestamps and have limited validity periods.
- Kerberos uses time information to help prevent replay of old authentication messages.
- If the client, KDC, and service server have significantly different clocks, a legitimate ticket may appear expired, not yet valid, or outside the permitted clock-skew window.
- This causes authentication failures even when the user's password and ticket are otherwise correct.
- Frequent clock differences can therefore reduce both availability and reliability of the authentication service.

2. Security Consequences

- Legitimate users may be denied access to required services.

- Automated services that depend on Kerberos may fail, affecting distributed applications.
- Administrators may be tempted to increase the acceptable clock-skew value excessively to hide synchronization problems; this can weaken the protection provided by time-limited tickets.
- Incorrect time can also complicate security-log correlation and incident investigation.

Recommended Solutions

- Deploy a reliable Network Time Protocol (NTP) or equivalent time-synchronization service across all Kerberos clients, KDCs, and service servers.
- Use trusted and appropriately secured time sources.
- Configure systems to synchronize automatically rather than relying on manual clock changes.
- Monitor clock offset and generate alerts when systems approach the allowed Kerberos clock-skew limit.
- Ensure time-zone configuration is correct and use a consistent time basis for system services.
- Avoid simply increasing the Kerberos clock-skew tolerance as the primary solution; correct the underlying synchronization problem instead.
- Protect time servers and synchronization traffic because an attacker who manipulates system time may disrupt authentication and security monitoring.
- After synchronization is restored, verify ticket acquisition, ticket validation, and service access across representative clients and servers.

Maintaining Security

A secure Kerberos environment should combine accurate time synchronization with secure KDC administration, strong credentials, appropriate ticket lifetimes, logging, monitoring, and controlled administrative access. Time synchronization should be treated as a security dependency rather than only a convenience for system administration.

Conclusion:

Kerberos depends on reasonably synchronized clocks because authentication tickets are time-sensitive. Reliable NTP-based synchronization, monitoring, secure time sources, and appropriate clock-skew configuration can restore reliable authentication without weakening the protocol's replay-resistance mechanisms.

QUESTION 4 – CERTIFICATE AUTHORITY BREACH IN PKI SYSTEM

A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains. Evaluate the impact of this breach on system trust and secure communication. Propose mechanisms to detect and mitigate such attacks in a PKI environment.

Answer

A Certificate Authority is a trusted component of a Public Key Infrastructure (PKI). It issues digital certificates that bind identities, such as domain names, to public keys. If a trusted CA is compromised and an attacker obtains fraudulent certificates for legitimate domains, the foundation of certificate-based trust can be seriously weakened.

Impact on System Trust

- A fraudulent certificate may cause a user's browser or application to believe that an attacker-controlled public key belongs to a legitimate domain.
- The attacker may use the certificate to impersonate a trusted service.
- Users can potentially be exposed to man-in-the-middle attacks if network traffic is redirected to the attacker's infrastructure.
- The breach undermines confidence in the affected CA and potentially in the certificate chains that depend on it.
- Organizations may need to determine whether certificates issued during the compromise period can still be trusted.

Impact on Secure Communication

TLS and similar protocols depend on certificate validation to authenticate endpoints. If a fraudulent certificate is accepted as valid, encryption may still be present but the

communicating party may be the wrong party. This demonstrates an important principle: confidentiality provided by encryption is not sufficient if endpoint authentication has been compromised.

Detection Mechanisms

- Monitor Certificate Transparency (CT) logs for unexpected certificates issued for organizational domains.
- Use certificate inventories and automated monitoring to identify certificates that were not authorized by the organization.
- Use appropriate certificate-status checking mechanisms such as CRLs and OCSP.
- Monitor DNS, TLS certificates, and domain infrastructure for unexpected changes.
- Use security monitoring and incident-response procedures to investigate suspicious certificates.
- Deploy browser, application, or enterprise controls that can detect unexpected certificate chains or unauthorized certificate issuances where appropriate.

Mitigation Mechanisms

- Revoke fraudulent certificates as quickly as possible.
- If the CA itself can no longer be trusted, remove or distrust the affected CA from relevant trust stores according to the incident-response plan.
- Issue legitimate replacement certificates through a trusted CA.
- Protect CA signing keys using strong access controls and hardware-backed security mechanisms.
- Separate CA roles and apply least privilege to reduce the impact of a single compromised account or component.
- Maintain offline or highly protected root CA keys where the PKI architecture supports such separation.

- Use multi-person approval for sensitive CA operations.
- Continuously monitor certificate issuance and investigate unexpected certificates.

The central risk is loss of trust in the identity-binding function of PKI. The most effective response combines immediate certificate revocation and trust remediation with stronger CA key protection, continuous certificate monitoring, Certificate Transparency visibility, and disciplined PKI governance.

Conclusion:

A compromised CA can enable attackers to obtain fraudulent certificates and impersonate trusted domains. Rapid detection, certificate revocation, trust-store remediation, protected CA keys, strong operational controls, and continuous monitoring are essential to restore confidence in the PKI environment.

QUESTION 5 – INTEGRITY VERIFICATION IN CLOUD DATA STORAGE

A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices. Analyze how incorrect implementation of hashing can lead to integrity failures. Recommend a robust integrity verification mechanism and justify its effectiveness.

Answer

Hash functions can detect accidental or unauthorized changes when a trusted reference hash is available and the comparison is performed correctly. However, simply calculating a hash is not sufficient to guarantee integrity. If the reference hash can also be modified by an attacker, or if the application validates the wrong data, uses an inappropriate algorithm, or performs the comparison incorrectly, tampering may go undetected.

How Incorrect Hash Validation Causes Integrity Failures

- If the expected hash is stored in the same unprotected location as the file, an attacker who changes the file may also replace the stored hash.
- If the application calculates the hash over the wrong file, incomplete data, or an unintended representation, changes to the actual content may not be detected correctly.
- Using weak or obsolete hash algorithms for new security designs can create unnecessary security risks.
- If the application does not verify the returned data before use, corrupted or tampered content may be accepted.

- Improper string encoding, file normalization, or inconsistent processing can result in mismatched hashes even when data is unchanged, or can cause incorrect validation logic.
- If hash comparison is implemented incorrectly, the system may accept a value that should have been rejected.
- A plain hash does not authenticate who created the expected hash. An attacker who can modify both the file and the reference hash can potentially defeat simple hash-based validation.

Recommended Robust Integrity Mechanism

For a security-sensitive cloud storage system, integrity should be based on a cryptographically strong hash combined with an authenticated integrity mechanism. A practical design is to use HMAC with a strong hash function such as SHA-256, with the secret HMAC key protected separately from the stored data. For higher-level workflows involving origin authentication and non-repudiation, digital signatures can also be used.

Recommended HMAC-Based Workflow

- When a trusted file is uploaded or accepted, calculate an HMAC over the exact file bytes using a securely stored secret key.
- Store the file and the resulting authentication tag separately from the secret key.
- When the file is retrieved, calculate the HMAC again over the exact retrieved bytes using the trusted secret key.
- Compare the newly calculated tag with the stored trusted tag using a secure comparison operation.
- Reject the file and raise an alert if the values do not match.
- Protect and rotate HMAC keys using a secure key-management system, and restrict access to the keys using least privilege.

- Maintain audit logs for uploads, downloads, integrity failures, and key-management events.

When Digital Signatures Are Preferable

If multiple independent parties need to verify integrity and origin without sharing a common secret, digital signatures are preferable. The data is hashed and the hash is signed using a private key. Verifiers use the corresponding public key to validate the signature. This can provide strong integrity, origin authentication, and, when implemented with appropriate key control and evidence, non-repudiation.

Justification

- HMAC prevents an attacker who lacks the secret key from generating a valid authentication tag for modified content.
- A strong cryptographic hash provides strong change detection for the protected data.
- Separating the key from the stored file and tag prevents a simple attacker who can modify storage contents from recomputing a valid HMAC.
- Digital signatures are suitable when verification must be performed by parties that should not possess a shared secret.
- Audit logging, access control, key management, and monitoring strengthen the overall integrity architecture.

Conclusion:

The cloud provider's problem is not simply the use of hashing, but the improper trust model and validation process. A robust integrity system should use a strong cryptographic construction such as HMAC-SHA-256 with protected keys, or digital signatures where public verification and stronger evidence of origin are required. The reference authentication value must be protected independently from the data being verified.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand